



**VEL TECH HIGH TECH  
Dr. RANGARAJAN Dr. SAKUNTHALA ENGINEERING  
COLLEGE**

*An Autonomous Institution*

Approved by AICTE-New Delhi, Affiliated to Anna University, Chennai  
Accredited by NBA, New Delhi & Accredited by NAAC with "A" Grade & CGPA of 3.27

**DEPARTMENT OF CSE(AIML)- Innovative Assignment 1**

|                          |                                        |                                 |                    |
|--------------------------|----------------------------------------|---------------------------------|--------------------|
| <b>Course Code</b>       | <b>25ML35T</b>                         | <b>Faculty Name</b>             | Dr.Manoj Kumar D S |
| <b>Course Name</b>       | Foundations of Artificial Intelligence | <b>Year / Sem / Sec</b>         | II / III / A       |
| <b>Name<br/>REG.NO:-</b> | Darshini.S<br>113025148016             | <b>BATCH.NO:-<br/>VHNO.NO:-</b> | 06<br>VH15177      |

**1.Problem Statement:- Campus Route Finder using BFS · BFS**

- **Build:** Find the shortest route between two campus locations on an unweighted graph using breadth-first search.
- **Requirements:** A defined graph; BFS implementation; printed/visualised path and hop count.
- **Must demonstrate:** Why BFS gives the fewest-edge path on an unweighted graph.

**2.Problem Formulation:-** The Campus Route Finder models a campus as an unweighted graph. Each campus location is represented as a vertex and each direct connection between two locations is represented as an undirected edge. Every edge has the same cost of one hop. The objective is to find a route from a user-selected starting location to a destination containing the minimum possible number of hops.

**Search formulation:-**

- **State:** A campus location (vertex) together with the information needed to reconstruct how that location was reached.
- **Actions:** Move from the current location to any directly connected neighbouring location.
- **Initial state:** The location entered by the user as the start location.
- **Goal test:** The current location is the destination entered by the user.
- **Path cost:** One unit for every connection traversed; therefore the total cost equals the number of hops.

The implemented graph contains nine named locations: Main Gate, Library, CSE Block, AI Lab, Canteen, Admin, Block, Auditorium, Hostel and Medical Center. Medical Center is intentionally isolated so that the no-route condition can be test

```
=====
[      CAMPUS ROUTE FINDER
      BFS Shortest Path
=====

Campus graph created successfully.
[
Available Locations:
- Auditorium
- Library
- CSE Block
- Medical Center
- Hostel
[- AI Lab
- Main Gate
- Canteen
- Admin Block

[Enter start location: Main Gate
Enter destination: Auditorium
```

**3.Approach: Breadth-First Search:-** BFS was selected because all campus connections have equal cost. BFS explores the graph level by level, beginning with the start location. Locations at level 1 are one hop away, level 2 are two hops away, and so forth. Consequently, when the destination is first reached, no path with fewer edges can exist.

The implementation is built from first principles rather than using a library search routine. The BFS class uses three key data structures:

- **Queue:**Maintains the frontier in first-in, first-out order so that locations are explored level by level.
- **Visited set:**Prevents the same location from being processed repeatedly.
- **Parent map:**Records the predecessor of each discovered location and allows the final route to be reconstructed.

The process starts by placing the start location in the queue and marking it visited. The algorithm repeatedly removes the next location, examines its neighbours, and records each newly discovered neighbour. Once the destination is found, the parent links are followed backwards to reconstruct the route, which is then reversed. The hop count is calculated as the number of locations in the route minus one.

**Why BFS is optimal here?** Suppose a destination was first reached using  $k$  edges. BFS has already completely explored every level from 0 through  $k-1$  before reaching level  $k$ . Therefore, a route containing fewer than  $k$  edges would have reached the destination earlier. Hence the first discovered route has the minimum number of edges for an unweighted graph.

**3. System Design and Implementation:-** The program is separated into four Java classes, giving each class a focused responsibility.

| Class              | Responsibility                                                                                |
|--------------------|-----------------------------------------------------------------------------------------------|
| <b>Location</b>    | Represents a campus location and stores its neighbouring locations.                           |
| <b>CampusGraph</b> | Stores locations and connections; supports adding, finding and retrieving locations.          |
| <b>BFS</b>         | Performs breadth-first search and reconstructs the shortest route using a parent map.         |
| <b>Main</b>        | Provides the command-line interface, accepts user input and displays the route and hop count. |

```

=====
[ CAMPUS ROUTE FINDER
[ BFS Shortest Path
=====

Campus graph created successfully.

Available Locations:
- Auditorium
- Library
- CSE Block
- Medical Center
- Hostel
- AI Lab
- Main Gate
- Canteen
- Admin Block

Enter start location: Main Gate
Enter destination: Auditorium

Shortest Route:
[Main Gate]
  ↓
[Canteen]
  ↓
[Admin Block]
  ↓
[Auditorium]

Hop Count: 3

```

The command-line program also normalizes location lookup to lowercase, allowing case-insensitive location names. It handles invalid locations, the same start and destination, and destinations for which no route exists.

**4. Results and Testing:-** The implementation was compiled with JDK 17+ and tested using the project test class. The following five cases cover normal shortest-path behaviour, the zero-hop case, reverse traversal, a longer reachable route, and an unreachable destination.

| Test case                  | Expected | Observed | Result |
|----------------------------|----------|----------|--------|
| Main Gate → Auditorium     | 3 hops   | 3 hops   | PASS   |
| Main Gate → Main Gate      | 0 hops   | 0 hops   | PASS   |
| Auditorium → Main Gate     | 3 hops   | 3 hops   | PASS   |
| Main Gate → Medical Center | No route | No route | PASS   |
| Main Gate → Hostel         | 4 hops   | 4 hops   | PASS   |

```

myswan@darshinis-MacBook-Air campus-route-finder % javac -d out src/*.java tests/BFSTest.j
ava
myswan@darshinis-MacBook-Air campus-route-finder % java -cp out BFSTest
PASS: Main Gate -> Auditorium | Hops = 3
PASS: Main Gate -> Main Gate | Hops = 0
PASS: Auditorium -> Main Gate | Hops = 3
PASS: Main Gate -> Hostel | Hops = 4
PASS: Main Gate -> Medical Center | No route found

```

**5. Complexity Analysis:-** For a graph with **V vertices** and **E edges**, BFS runs in  **$O(V + E)$**  time when using an adjacency-list representation. Each reachable vertex is visited at most once and each relevant edge is examined during neighbour traversal. The queue, visited set and parent map require  **$O(V)$**  auxiliary space. For this small fixed campus graph, execution is effectively instantaneous and the main practical limitation is not computational cost but graph maintenance: connections must be explicitly defined in the source code. The current model also treats every connection as equal, so it cannot distinguish a short physical path from a long one when both count as one graph edge.

## **6. Limitations and Possible Improvements:-**

- The graph is manually defined rather than loaded from a campus map or data file.
- All edges have equal cost; real walking distance, time, accessibility and obstacles are not represented.
- The current interface is command-line based rather than a graphical or web map.
- A future version could use weighted edges with an appropriate shortest-path algorithm, add a graphical campus map, and allow the user to edit or load the graph dynamically.

**7. Reflection and Conclusion:-** The main implementation challenge was translating the theoretical BFS process into a complete application: modelling the campus as an undirected graph, avoiding repeated visits, preserving parent relationships, reconstructing the path and converting the resulting path into a hop count. Testing edge cases was also important because a route finder should not assume that every requested destination is reachable.

The completed project demonstrates how an uninformed search algorithm can solve a practical navigation problem when the objective is the minimum number of edges. The results confirm the expected BFS behaviour on the defined campus graph, including shortest-path selection, reverse traversal, zero-hop traversal and the absence of a route to an isolated location.

**8.Honesty / source note:-** AI assistance was used during development for guidance on project structure, documentation and explanation; the submitted implementation and results were tested by the Author(Darshini.S).

**9. Repository deliverable:-**The project is maintained as a public GitHub repository containing source code, tests, README, dependencies, .gitignore and LICENSE.

```
Hop Count: 3
[myswan@darshinis-MacBook-Air campus-route-finder % find . -not -path './.git/*' -not -path './out/*' -print
.
./LICENSE
./out
./tests
./tests/BFSTest.java
./docs
./README.md
./dependencies.txt
./gitignore
./.git
./src
./src/BFS.java
./src/CampusGraph.java
./src/Main.java
./src/Location.java
```